

Smart Invoice & Payment Tracking

API Backend — Gestion de Factures Fournisseurs

Anass Kabil

Node.js

Express

MongoDB

JWT

Jury Blanc
avril-2026

Le Projet en Un Coup d'Oeil

Le Problème

- Suivre des dizaines de factures fournisseurs est complexe
- Difficile de savoir quelles factures sont en retard
- Pas de vision claire des montants payés / restants
- Gestion manuelle = erreurs et perte de temps

Notre Solution

- ✓ API backend centralisée et sécurisée
- ✓ Suivi automatique du statut de chaque facture
- ✓ Paiements partiels ou complets enregistrés
- ✓ Accès personnel : chaque client voit uniquement ses données
- ✓ Statistiques et tableau de bord en temps réel

Technologies Utilisees

Node.js

Environnement d'exécution JavaScript côté serveur. Fait tourner l'API en dehors du navigateur.

Express.js

Framework pour créer les routes de l'API (les adresses auxquelles on envoie des requêtes).

MongoDB

Base de données NoSQL (documents JSON). Stocke utilisateurs, fournisseurs, factures et paiements.

Mongoose

Définit la structure et les règles de validation des données avant de les sauvegarder en base.

JWT

JSON Web Token — badge numérique délivré à la connexion, prouve l'identité sur chaque requête.

bcrypt

Transforme le mot de passe en empreinte numérique illisible avant de le stocker.

Structure du Projet

```
gestionFactures/  
├── server.js  
├── config/  
│   └── db.js  
├── models/  
│   ├── User.js  
│   ├── Suppliers.js  
│   ├── Invoice.js  
│   └── Payment.js  
├── controllers/  
│   ├── auth.controller  
│   ├── suppliers.controller  
│   ├── invoice.controller  
│   └── payment.controller  
├── routes/  
│   ├── auth.routes  
│   ├── suppliers.routes  
│   └── invoices.routes  
└── middlewares/  
    └── auth.middleware
```

server.js

Point d'entree — démarre le serveur et connecte toutes les pieces

config/

Connexion a la base de donnees MongoDB

models/

Structure des donnees (utilisateurs, factures...)

controllers/

Logique metier — ce qui se passe quand une requete arrive

routes/

Adresses URL disponibles et quel controller les traite

middlewares/

Verification du token JWT avant chaque requete protegee

Comment ça Fonctionne ?

1

Inscription

Création d'un compte avec email et mot de passe. Le mot de passe est haché (bcrypt) avant d'être sauvegarde.

2

Connexion

L'API vérifie les identifiants et délivre un token JWT valable 7 jours. Ce badge est requis pour toute action.

3

Fournisseurs

Le client enregistre ses fournisseurs (entreprises chez qui il achète) : nom, email, telephone...

4

Factures

Pour chaque achat, il creer une facture : montant, date d'echance, fournisseur lie.

5

Paielements

Il paye en une ou plusieurs fois. Le système met a jour le statut automatiquement.

Securite & Controle d'Acces



Hachage du mot de passe (bcrypt)

Le mot de passe n'est jamais stocké tel quel. Il est transformée en empreinte numerique — même l'admin ne peut pas le lire.



Token JWT a la connexion

Apres connexion, l'utilisateur reçoit un token (badge). Il doit le présenter a chaque requête dans le header : Authorization: Bearer {token}.



Middleware d'authentification

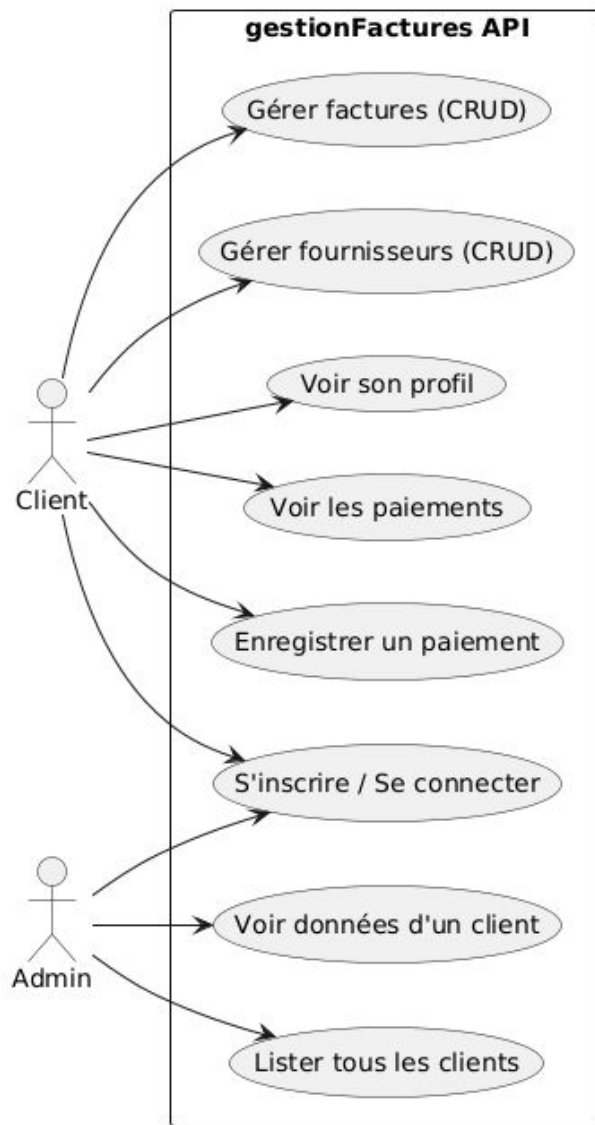
Avant chaque action protegee, le serveur verifie la validité du token. Si absent ou invalide, la requete est bloquee (401).



Isolation des donnees

Chaque donnée est liée a son propriétaire. Un utilisateur ne peut jamais lire ou modifier les données d'un autre.

Diagramme de Cas d'Utilisation



Interpretation

Client (entreprise / freelance)

- Créer son compte et se connecte
- Gere ses fournisseurs (ajouter, modifier, supprimer)
- Cree et suit ses factures
- Enregistre des paiements (partiels ou totaux)
- Consulte ses statistiques

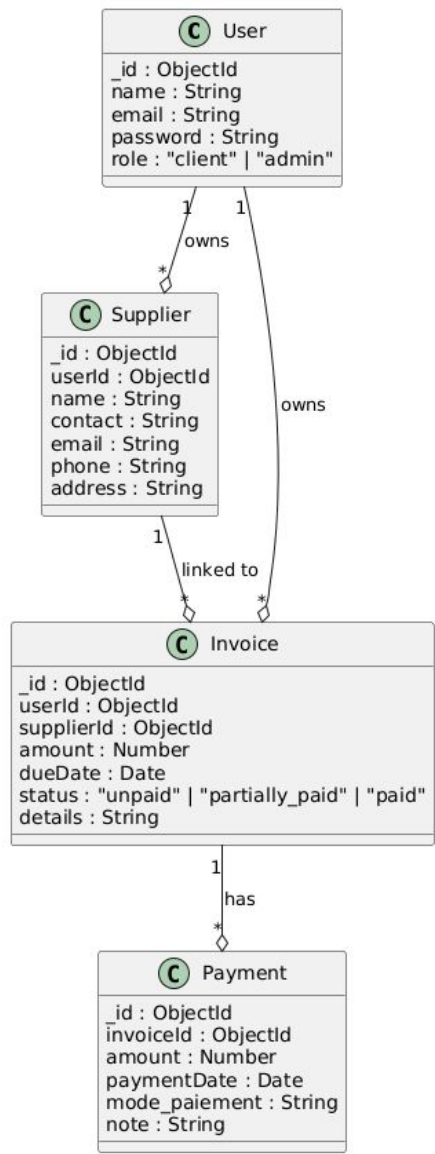
Admin (administrateur de plateforme)

- Visualise tous les clients inscrits
- Consulte les données de n'importe quel client

Chaque role a des droits distincts :

- Le client ne voit que ses propres données.
- L'admin a une vue globale en lecture seule.

Diagramme de Classes



Ce que represente ce diagramme

User

L'utilisateur du systeme. Il peut etre 'client' (entreprise) ou 'admin'. Chaque donnee lui appartient.

Supplier

Un fournisseur cree par le client. Il appartient a un seul client et peut avoir plusieurs factures.

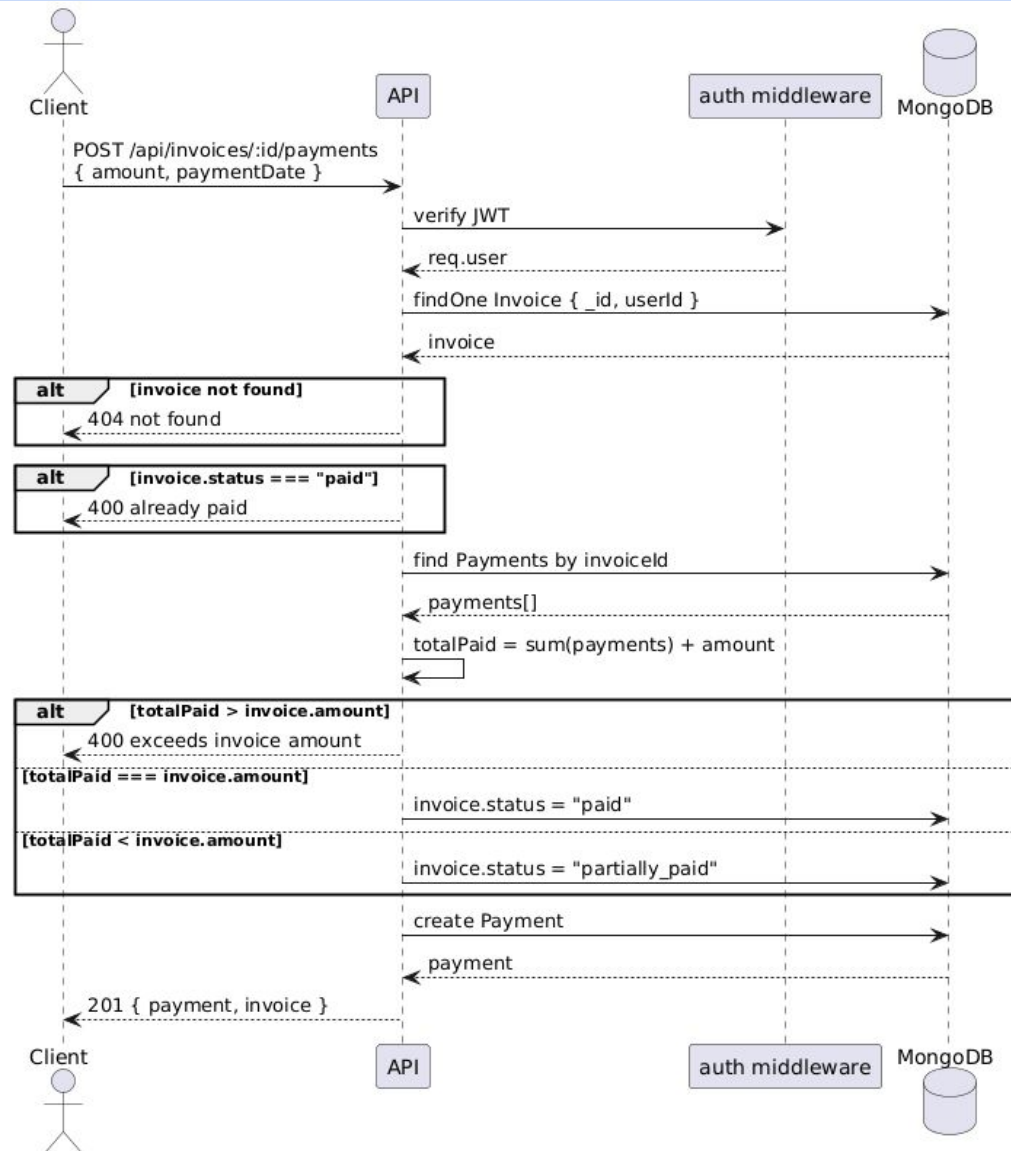
Invoice

Une facture liee a un fournisseur. Elle a un montant, une date limite et un statut mis a jour automatiquement.

Payment

Un paiement lie a une facture. Plusieurs paiements partiels peuvent exister pour une meme facture.

Diagramme de Séquence — Enregistrer un Paiement



Lecture du diagramme

- 1. Requête** L'utilisateur envoie le montant et la date de paiement.
- 2. JWT** Le middleware controle le token. Invalide → acces refuse.
- 3. Facture** L'API cherche la facture. Introuvable → erreur 404.
- 4. Statut** Facture deja 'payee' → ajout refuse immediatement.
- 5. Calcul** Somme des paiements existants + nouveau montant.
- 6. Mise a jour** Statut mis a jour automatiquement et sauvegarde.
- 7. Réponse** Le paiement et la facture mis a jour sont retournes.

Les Routes de l'API

Authentification

POST

Créer un compte

/api/auth/register

POST

Connexion → token JWT

/api/auth/login

GET

Voir son profil

/api/auth/me

Fournisseurs

POST

Ajouter un fournisseur

/api/suppliers

GET

Lister ses fournisseurs

/api/suppliers

PUT

Modifier

/api/suppliers/:id

DELETE

Supprimer

/api/suppliers/:id

Factures

POST

Créer une facture

/api/invoices

GET

Lister ses factures

/api/invoices

PUT

Modifier (si non payee)

/api/invoices/:id

DELETE

Supprimer (si sans paiement)

/api/invoices/:id

Paielements

POST

Enregistrer un paiement

/api/invoices/:id/payments

GET

Lister les paiements

/api/invoices/:id/payments

Suivi Automatique du Statut des Factures

UNPAID

Non payee

Aucun paiement n'a été enregistré pour cette facture.

La facture vient d'être créée ou aucun versement n'est arrivé.

PARTIAL

Partiellement payee

Un ou plusieurs paiements ont été reçus, mais le total n'atteint pas encore le montant de la facture.

Ex : facture de 1000 MAD, on a payé 600 MAD → statut partially_paid.

PAID

Totalement payee

La somme de tous les paiements est égale au montant total de la facture.

Une fois ce statut atteint, la facture ne peut plus être modifiée ni payée davantage.

Ce qui a été réalisé

- ✓ Authentification complete (inscription, connexion, profil) securisee avec JWT + bcrypt
- ✓ CRUD complet pour les fournisseurs — chaque client gère uniquement les siens
- ✓ CRUD complet pour les factures avec contrôles métier (modification / suppression)
- ✓ Système de paiements avec contrôle de montant et mise a jour automatique du statut
- ✓ Isolation totale des donnees : aucun utilisateur ne voit les données d'un autre
- ✓ Health check + gestion des erreurs coherente (codes HTTP appropries)
- ✓ Diagrammes UML (cas d'utilisation, classes, sequence)

Stack Technique

Node.js + Express

MongoDB + Mongoose

JWT + bcrypt

Architecture MVC

REST API